

10.2: Streaming and Real-time Analytics



UMD DATA605 - Big Data Systems

10.2: Streaming and Real-time Analytics

– **Instructor:** Dr. GP Saggese, gsaggese@umd.edu

1 / 21

2 / 21: Data Streams: Motivation

Data Streams: Motivation

- Big Data is generated as a **continuous, unbounded stream**
- Applications generate data at **high velocity**
 - Financial transactions and market feeds
 - Sensor instrumentation, RFID, IoT telemetry
 - Network and system monitoring
 - Continuous media (video, audio)
- A **data stream** is a time-ordered sequence of events
 - Stream processing treats streams as first-class computational objects
- **Requirements**
 - Ingest and handle high-throughput event streams
 - Low-latency, near-real-time operations (e.g., time-series analytics)
 - Efficient dissemination of relevant subsets to consumers
 - Distributed processing to scale beyond a single machine

2 / 21

- **Data Streams: Motivation**
 - *Big Data is generated as a continuous, unbounded stream*
 - In today's world, data is not just stored in static databases. Instead, it flows continuously like a river. This means that data is always being generated and never stops, which is why we call it "unbounded."
- **Applications generate data at high velocity**
 - *Financial transactions and market feeds:* Stock markets and banks produce a lot of data every second as people buy and sell.
 - *Sensor instrumentation, RFID, IoT telemetry:* Devices like sensors and IoT gadgets send data constantly, such as temperature readings or location updates.
 - *Network and system monitoring:* Computers and networks are always being checked for performance and security, creating a steady stream of data.
 - *Continuous media (video, audio):* Streaming services like Netflix or Spotify send data continuously as you watch or listen.
- *A data stream is a time-ordered sequence of events*
 - When we talk about data streams, we mean that the data comes in a specific order based on time. Stream processing is a way to handle this data as it comes in, treating it as important information that needs to be processed immediately.
- **Requirements**
 - *Ingest and handle high-throughput event streams:* Systems need to be able to take in and process large amounts of data quickly.
 - *Low-latency, near-real-time operations (e.g., time-series analytics):* We need to process data almost instantly to make timely decisions, like analyzing trends over time.
 - *Efficient dissemination of relevant subsets to consumers:* Not all data is useful to every-

one, so systems must efficiently send the right data to the right people or applications.

- *Distributed processing to scale beyond a single machine:* To handle the vast amount of data, we need to use multiple computers working together, as one machine isn't enough.

3 / 21: Data Streams: Examples

Data Streams: Examples

- Continuous queries
 - Any SQL query can be continuous
 - E.g., “compute moving average over last hour every 10 mins”
- Anomaly detection, pattern recognition
 - E.g., “alert me when A occurs and then B within 10 mins”
 - Correlate events from different streams
- Statistical tasks
 - E.g., de-noising measured readings
 - Build an online machine learning model
- Process multimedia data
 - E.g., online object detection, activity detection

3 / 21

– Data Streams: Examples

– Continuous queries

- Continuous queries are like regular SQL queries but designed to run continuously over time. This means they keep processing data as it arrives, rather than just once.
- For example, a query like “compute moving average over last hour every 10 mins” would continuously update the moving average every 10 minutes using the most recent hour of data. This is useful for real-time monitoring and decision-making.

– Anomaly detection, pattern recognition

- These tasks involve identifying unusual patterns or sequences in data streams. For instance, you might want to “alert me when A occurs and then B within 10 mins”, which means you are looking for a specific sequence of events within a short time frame.
- This often requires correlating events from different data streams to detect complex patterns or anomalies, which can be crucial for security or operational efficiency.

– Statistical tasks

- Statistical tasks in data streams might involve cleaning or improving the quality of incoming data, such as de-noising measured readings to remove random fluctuations.
- Building an online machine learning model means updating the model continuously as

new data comes in, allowing it to adapt to changes over time and improve its predictions or classifications.

– **Process multimedia data**

- Processing multimedia data in real-time can involve tasks like online object detection or activity detection. This means analyzing video or audio streams to identify objects or actions as they happen.
- These tasks are computationally intensive and require efficient algorithms to handle the high volume and velocity of multimedia data.

4 / 21: Why Not Using Standard Solutions?

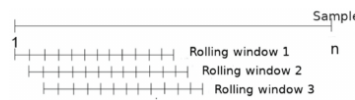
Why Not Using Standard Solutions?

– **Example**

- “Report moving average of XYZ over the last hour every 10 minutes”

– **Solution**

- Insert arriving items into a relational table
- Re-run query repeatedly



$$m_n = \frac{1}{n} \sum_{i=1}^n a_i$$

– **Problems**

- Re-executes full query, not leveraging incremental updates
- Many streaming computations are recursive
- Complex computations may not be easily expressed incrementally
- Real systems may run thousands of continuous queries

$$m_n = m_{n-1} + \frac{a_n - m_{n-1}}{n}$$

4 / 21

– **Example**

- The task is to report the moving average of a variable, XYZ, over the last hour, updating every 10 minutes. This is a common requirement in data analysis where real-time insights are needed.

– **Solution**

- A typical approach might involve inserting new data into a relational database table and then re-running the query to calculate the moving average. This method is straightforward but not efficient for real-time data processing.

– **Problems**

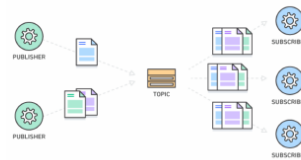
- **Re-execution of Full Query:** Each time the query runs, it processes all data from scratch, which is inefficient and time-consuming.
- **Lack of Incremental Updates:** The solution doesn't take advantage of incremental updates, which could save resources by only processing new data.
- **Recursive Computations:** Many streaming computations require recursive logic,

- which is not easily handled by standard SQL queries.
- **Complexity:** Some computations are too complex to express incrementally in a relational database.
- **Scalability:** Real-world systems often need to handle thousands of continuous queries, which can overwhelm traditional database systems.
- **Streaming Concepts**
 - **Rolling Windows:** These are used to calculate metrics like moving averages over a specific time frame, updating as new data comes in.
 - **Mathematical Formulas:**
 - The first formula represents the basic calculation of a moving average.
 - The second formula shows an incremental approach, updating the moving average as new data arrives, which is more efficient for streaming data.

5 / 21: Pub-Sub Systems: Motivation

Pub-Sub Systems: Motivation

- Modern distributed systems use **small, independent components**
 - E.g., serverless architectures, microservices (e.g., Uber)
 - Easier evolution, isolation, scalability
- **Publish-subscribe (pub-sub) systems**
 - Aka “message queues”, “message brokers”
 - Connect producers and consumers for event distribution
 - Topics cluster related messages
 - Typically provide lightweight dissemination rather than complex queries
 - Examples: AWS SQS, Kinesis, Kafka, RabbitMQ, Redis Streams, Celery, JBoss



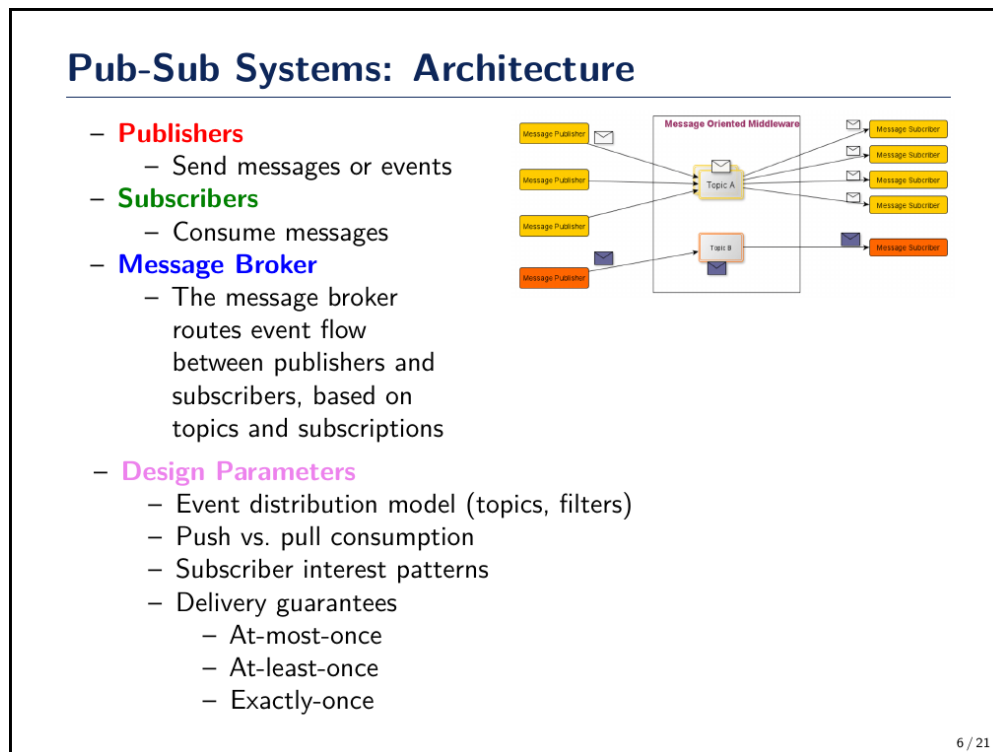
5 / 21

- **Modern distributed systems use small, independent components**
 - *Serverless architectures* and *microservices* are examples of this approach.
 - These systems, like those used by companies such as Uber, allow for easier evolution, isolation, and scalability.
 - By breaking down applications into smaller parts, each component can be developed, deployed, and scaled independently.
- **Publish-subscribe (pub-sub) systems**
 - Also known as “message queues” or “message brokers.”
 - These systems connect producers (publishers) and consumers (subscribers) to facilitate event distribution.
 - Messages are grouped into *topics*, which cluster related messages together.

- Pub-sub systems typically focus on lightweight message dissemination rather than complex querying.
- Examples of pub-sub systems include AWS SQS, Kinesis, Kafka, RabbitMQ, Redis Streams, Celery, and JBoss.
- These systems are crucial for enabling real-time data processing and communication between distributed components.

The accompanying image illustrates how publishers send messages to a central topic, which are then distributed to multiple subscribers, highlighting the flow and organization of messages in a pub-sub system.

6 / 21: Pub-Sub Systems: Architecture



- **Publishers**
 - *Role:* Publishers are responsible for sending messages or events into the system. They act as the source of information, generating data that needs to be communicated to interested parties.
- **Subscribers**
 - *Role:* Subscribers consume messages. They are the recipients who express interest in certain types of messages or events and receive them from the system.
- **Message Broker**
 - *Function:* The message broker is a crucial component that routes the flow of events between publishers and subscribers. It uses topics and subscriptions to determine how messages are distributed, ensuring that each subscriber receives the messages they are interested in.
- **Design Parameters**
 - **Event Distribution Model:** This involves organizing how messages are categorized

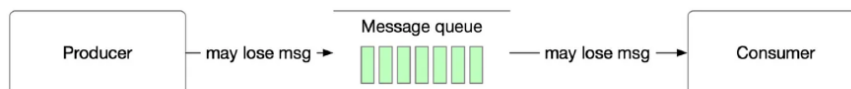
and filtered. Topics are used to group messages, while filters can refine which messages are sent to which subscribers.

- **Push vs. Pull Consumption:** This refers to how messages are delivered to subscribers. In a push model, messages are sent automatically, while in a pull model, subscribers request messages when they are ready to process them.
- **Subscriber Interest Patterns:** This involves understanding and managing the different ways subscribers express interest in messages, which can affect how messages are routed and delivered.
- **Delivery Guarantees:** These are assurances about how messages are delivered:
 - *At-most-once:* Messages are delivered no more than once, with the risk of some messages being lost.
 - *At-least-once:* Messages are delivered at least once, ensuring delivery but possibly resulting in duplicates.
 - *Exactly-once:* Messages are delivered exactly once, ensuring no duplicates and no losses, which is ideal but often more complex to implement.

7 / 21: Delivery Semantics: At-most once

Delivery Semantics: At-most once

- **At-most once:** *message may be lost, not redelivered*



- **Pros**
 - Small implementation overhead, high-performance
 - Easy to implement: “fire-and-forget”
- Works when occasional loss is acceptable
 - E.g., monitoring metrics of a website

7 / 21

- **Delivery Semantics: At-most once**
 - *At-most once* delivery means that a message might be lost and not delivered to the recipient. This is a common approach in systems where occasional data loss is acceptable.
- **Pros**
 - **Small implementation overhead, high-performance:** This method is efficient because it doesn’t require complex mechanisms to ensure message delivery, which can slow down the system.
 - **Easy to implement: “fire-and-forget”:** The simplicity of this approach lies in its

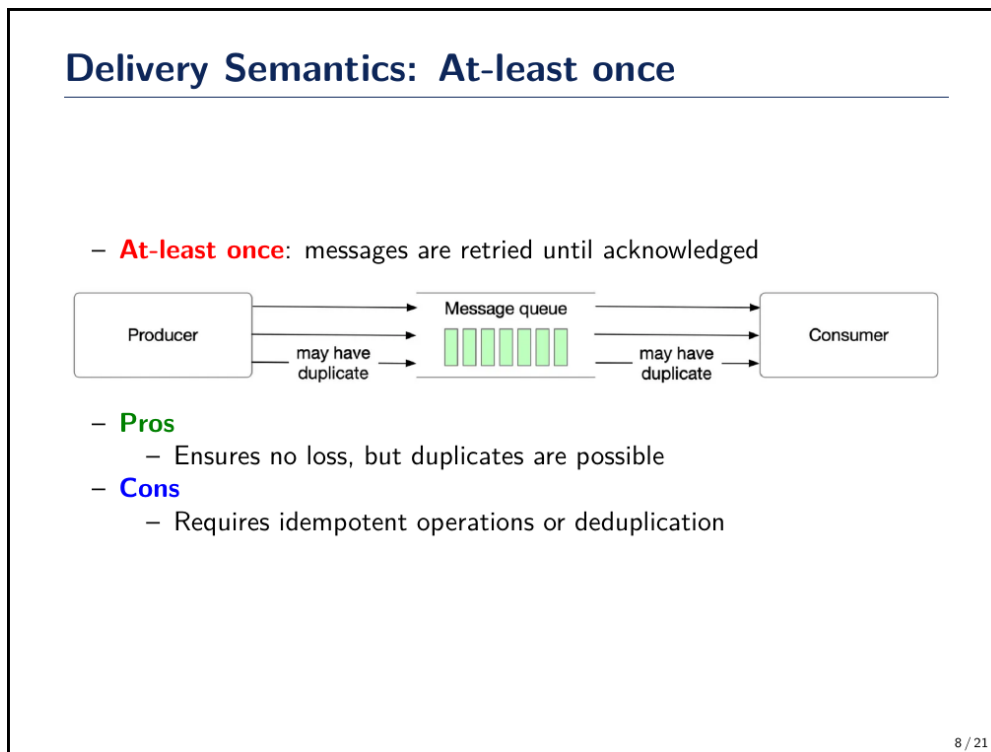
straightforwardness. Once a message is sent, the sender doesn't need to worry about its delivery status.

- **Works when occasional loss is acceptable**

- This approach is suitable for scenarios where losing some messages doesn't significantly impact the overall system. For example, in monitoring metrics of a website, missing a few data points might not be critical as long as the overall trend is captured.

The diagram illustrates a typical setup where a producer sends messages to a consumer through a message queue. In this setup, messages can be lost at any stage, either during transmission to the queue or from the queue to the consumer.

8 / 21: Delivery Semantics: At-least once



- **At-least once:** This delivery semantic ensures that messages are sent repeatedly until the sender receives an acknowledgment from the receiver. This approach is crucial in systems where message loss is unacceptable, as it guarantees that every message will eventually be delivered.

- **Pros**

- *Ensures no loss:* The primary advantage of the at-least-once delivery is that it prevents message loss. Every message will reach its destination, which is vital for applications where missing data could lead to significant issues.
- *Duplicates are possible:* While this method ensures delivery, it can result in the same message being delivered multiple times. This is because the system continues to send the message until it gets a confirmation, which might lead to duplicates if the acknowledgment is delayed or lost.

- **Cons**

- *Requires idempotent operations or deduplication:* To handle potential duplicates, systems must implement idempotent operations—processes that can be applied multiple times without changing the result beyond the initial application. Alternatively, deduplication strategies can be used to filter out repeated messages, ensuring that each message is processed only once.

9 / 21: Delivery Semantics: Exactly once

Delivery Semantics: Exactly once

- **Exactly once:** each message is processed once globally



- Most consumer-friendly but hardest to guarantee
 - Complicated by distributed coordination limits (e.g., “Two Generals’ Problem”)
- Used in financial and mission-critical systems
 - E.g., payment, trading, accounting systems

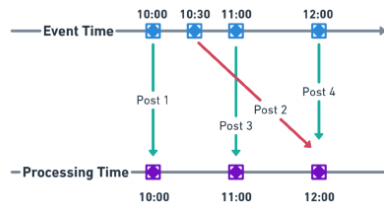
9 / 21

- **Exactly once:** This delivery semantic ensures that each message is processed only once across the entire system. It is the most reliable form of message delivery, preventing duplicates and ensuring data consistency.
- **Most consumer-friendly but hardest to guarantee:**
 - This approach is ideal for consumers because it eliminates the risk of processing the same message multiple times, which can lead to errors or inconsistencies.
 - However, achieving this level of reliability is challenging due to the complexities involved in coordinating distributed systems. A classic example of this complexity is the “Two Generals’ Problem,” which illustrates the difficulties in achieving consensus in distributed networks.
- **Used in financial and mission-critical systems:**
 - Systems that require high reliability and accuracy, such as payment processing, trading platforms, and accounting systems, often implement exactly-once semantics.
 - In these contexts, processing a message more than once or not at all can have significant financial or operational consequences, making exactly-once delivery essential.

10 / 21: Event vs Processing Time

Event vs Processing Time

- In both streaming and pub-sub architectures
 - **Event time**
 - Time when each record is generated
 - **Processing time**
 - Time when each record is received
 - Ingestion vs processing time: when events are received vs processed
- **Problems with events**
 - Events may arrive late or out of order
 - Determining how long to wait for stragglers is difficult
 - Systems set bounds on lateness
 - Extremely late data may be dropped or trigger re-computation



10 / 21

- **Event vs Processing Time**
 - In streaming and pub-sub architectures, understanding the difference between *event time* and *processing time* is crucial.
 - **Event time** refers to when each record is generated. This is the actual time the event occurs in the real world.
 - **Processing time** is when each record is received by the system. It includes the time taken for events to be ingested and processed, which can differ due to network delays or system load.
- **Problems with Events**
 - Events can arrive late or out of order, which complicates processing.
 - Determining how long to wait for late events, known as stragglers, is challenging.
 - Systems often set limits on how late data can be. Extremely late data might be ignored or cause the system to recompute results to maintain accuracy.
- **Apache Streaming Zoo Example**
 - The diagram illustrates the difference between event time and processing time.
 - Events (Post 1, Post 2, etc.) are generated at specific event times but are processed at different times due to delays.
 - This highlights the importance of handling late or out-of-order data in streaming systems.

11 / 21: Apache Streaming Zoo

Apache Streaming Zoo

- Many different streaming frameworks in the Apache family
 - E.g., Apex, Beam, Flink, Kafka, Spark, Storm, NiFi
 - Built simultaneously at different companies, then open-sourced
- **Different workloads**
 - Real-time analytics, continuous computation
 - Streaming ML, ETL pipelines
 - Messaging and log aggregation
- **Differences arise in**
 - Batch vs. streaming orientation
 - Delivery semantics
 - Compute vs. pub-sub roles
 - Throughput, latency, fault tolerance
 - API and language support

11 / 21

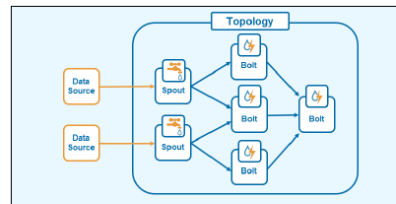
- **Apache Streaming Zoo**
 - The term “Apache Streaming Zoo” refers to the variety of streaming frameworks developed under the Apache Software Foundation. These frameworks are designed to handle data streams in real-time, providing tools for processing and analyzing data as it flows.
 - **Examples:** Some popular frameworks include *Apache Apex*, *Apache Beam*, *Apache Flink*, *Apache Kafka*, *Apache Spark*, *Apache Storm*, and *Apache NiFi*. Each of these was initially developed by different companies to meet specific needs before being released as open-source projects.
- **Different workloads**
 - These frameworks are used for a range of tasks. **Real-time analytics** involves analyzing data as it is generated, which is crucial for applications like monitoring systems. **Continuous computation** refers to ongoing data processing without interruption.
 - **Streaming ML** (Machine Learning) involves applying machine learning models to data streams, allowing for real-time predictions. **ETL pipelines** (Extract, Transform, Load) are used to process and move data between systems. **Messaging and log aggregation** involve collecting and organizing log data from various sources for analysis.
- **Differences arise in**
 - Frameworks differ in their approach to data processing. Some are more suited to **batch processing**, where data is processed in chunks, while others excel in **streaming**, handling data continuously.
 - **Delivery semantics** refer to how data is delivered and processed, affecting reliability and consistency.
 - The role of a framework can be more focused on **compute** (processing data) or **pub-sub** (publish-subscribe, managing data flow between producers and consumers).

- Key performance metrics include **throughput** (amount of data processed in a given time), **latency** (time taken to process data), and **fault tolerance** (ability to handle failures).
- **API and language support** varies, with some frameworks offering more flexibility in terms of programming languages and interfaces, which can influence ease of use and integration with other systems.

12 / 21: Apache Storm

Apache Storm

- Open-source distributed real-time computation system
 - Acquired and open-sourced by Twitter
- **Horizontal scalability**: add machines to handle increasing data
- **Directed acyclic graph (DAG)**:
 - Spouts as data sources (as source nodes)
 - Bolts as processing units (as nodes)
 - Data streams (as edges)
- **Fault tolerance**:
 - At-least-once processing
 - Automatic task restarts
 - Workload redistribution
- **Suitable for**
 - Complex data processing workflows
 - With multiple stages and parallelism



12 / 21

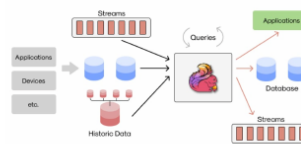
- **Apache Storm** is an open-source system designed for real-time computation. It was initially developed by Twitter, which later made it available to the public. This system is particularly useful for processing large streams of data in real-time.
- **Horizontal scalability** is a key feature of Apache Storm. This means you can add more machines to the system to handle increasing amounts of data, making it highly adaptable to growing data needs.
- **Directed Acyclic Graph (DAG)** is the structure used by Apache Storm to process data:
 - **Spouts** act as data sources, initiating the data flow.
 - **Bolts** are the processing units that perform computations on the data.
 - **Data streams** connect spouts and bolts, forming the edges of the graph.
- **Fault tolerance** is built into Apache Storm to ensure reliability:
 - It guarantees *at-least-once processing*, meaning data is processed at least once even if failures occur.
 - Tasks are automatically restarted if they fail.
 - The system can redistribute workloads to maintain performance.

- **Suitable** for complex data processing workflows, Apache Storm excels in scenarios requiring multiple stages and parallel processing, making it ideal for applications like real-time analytics and monitoring.

13 / 21: Apache Kafka

Apache Kafka

- Open-source distributed streaming platform
 - Developed at LinkedIn, open-sourced in 2011
- **Core components**
 - Producers
 - Brokers
 - Consumers
 - Topics
 - Partitions
- **Delivery**: at-least-once, at-most-once, exactly-once
- High throughput, low latency
 - Persistent, replicated log storage
- *Kafka Connect* for integration with external systems
- *Kafka Streams* for native stream processing



13 / 21

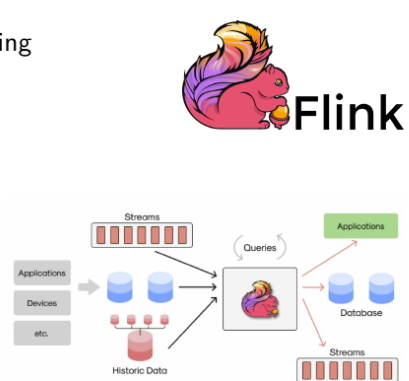
- **Open-source distributed streaming platform**
 - Apache Kafka is a powerful tool for handling real-time data feeds. It was originally developed at LinkedIn and became open-source in 2011. This means anyone can use, modify, and distribute it freely, which has contributed to its widespread adoption.
- **Core components**
 - **Producers**: These are the applications that send data to Kafka.
 - **Brokers**: These are the servers that store the data and serve it to consumers.
 - **Consumers**: These are the applications that read data from Kafka.
 - **Topics**: These are categories or feeds to which records are published.
 - **Partitions**: Each topic is split into partitions, allowing for parallel processing and scalability.
- **Delivery**: Kafka supports different delivery guarantees:
 - *At-least-once*: Ensures that messages are not lost but may be delivered more than once.
 - *At-most-once*: Ensures that messages are delivered at most once, but some may be lost.
 - *Exactly-once*: Ensures that messages are delivered exactly once, which is crucial for certain applications.
- **High throughput, low latency**
 - Kafka is designed for high throughput and low latency, making it suitable for handling large volumes of data quickly. It uses persistent, replicated log storage to ensure data durability and reliability.

- **Kafka Connect** for integration with external systems
 - This tool allows Kafka to connect with various data sources and sinks, making it easier to move data between Kafka and other systems.
- **Kafka Streams** for native stream processing
 - Kafka Streams is a client library for building applications and microservices, where the input and output data are stored in Kafka clusters. It allows for real-time processing of data streams.

14 / 21: Apache Flink

Apache Flink

- Open-source, distributed data processing framework
- Distributed processing engine with strong support for stateful streaming
- Exactly-once semantics via checkpointing and robust state management
- Unified API for batch and streaming
- Rich windowing functions
- Runs on standalone clusters, YARN, Mesos, Kubernetes, and cloud



The diagram illustrates the Apache Flink architecture. On the left, 'Applications', 'Devices', and 'etc.' feed into 'Streams' (represented by red bars). These streams feed into a central processing engine (a box with the Flink logo). The engine also receives 'Historic Data' (represented by a red cylinder). From the engine, 'Queries' (represented by a red cylinder) are sent to 'Applications' and 'Database' (represented by blue cylinders). The engine also outputs 'Streams' (represented by red bars) to the right. The Flink logo is shown above the diagram.

14 / 21

- **Apache Flink** is an open-source framework designed for distributed data processing. It excels in handling large-scale data streams and batch processing, making it a versatile tool for real-time analytics and data-driven applications.
- **Distributed Processing Engine:** Flink is built to efficiently process data across multiple nodes, providing strong support for *stateful streaming*. This means it can maintain and manage state information across streams, which is crucial for complex event processing.
- **Exactly-once Semantics:** Through checkpointing and robust state management, Flink ensures that each piece of data is processed exactly once, even in the event of failures. This reliability is essential for applications where data accuracy is critical.
- **Unified API:** Flink offers a unified programming model for both batch and streaming data, simplifying the development process and allowing developers to use the same codebase for different types of data processing tasks.
- **Rich Windowing Functions:** These functions allow users to define how data is grouped and processed over time, which is particularly useful for time-based analytics.

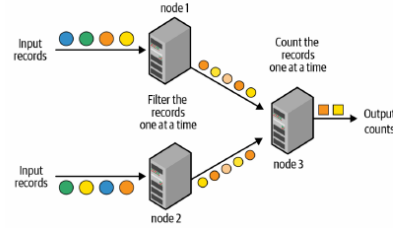
- **Deployment Flexibility:** Flink can run on various platforms, including standalone clusters, YARN, Mesos, Kubernetes, and cloud environments, providing flexibility in deployment and scalability.
- **Processing Styles:** The diagram illustrates how Flink integrates with various data sources and sinks, processing streams and queries to provide insights and drive applications. This highlights Flink's role in transforming raw data into actionable information.

15 / 21: Record-at-a-time Processing

Record-at-a-time Processing

- **Designed to handle infinite data streams**
 - Implemented in Apache Kafka
- **Distributed processing over multiple nodes**
 - Nodes organized in a DAG
 - Each node continuously:
 - Receives a single record
 - Processes the record immediately
 - Forwards the output to the next node
- **Pros**
 - Achieves extremely low latency
 - E.g., sub-millisecond response times
- **Cons**
 - Poor fault tolerance
 - Requires extra nodes or redundant paths for failover
 - Sensitive to stragglers
 - Slow nodes can delay the entire pipeline

15 / 21



The diagram illustrates a Directed Acyclic Graph (DAG) for record-at-a-time processing. It shows three nodes: node 1, node 2, and node 3. Node 1 and node 2 are the first stage, each receiving 'Input records' (represented by colored dots). Both nodes perform the operation 'Filter the records one at a time'. The filtered records are then sent to node 3, which performs 'Count the records one at a time' and finally outputs 'Output counts'.

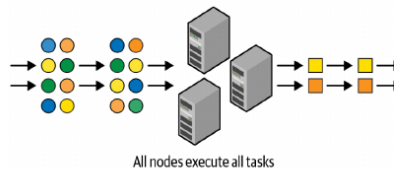
- **Record-at-a-time Processing**
 - *Designed to handle infinite data streams:* This approach is ideal for systems that need to process continuous flows of data without interruption. Apache Kafka is a popular platform that implements this method, allowing for real-time data processing.
- **Distributed processing over multiple nodes**
 - *Nodes organized in a DAG:* A Directed Acyclic Graph (DAG) structure is used to organize nodes, ensuring that data flows in a single direction without cycles.
 - Each node continuously:
 - *Receives a single record:* Nodes handle one piece of data at a time, ensuring immediate processing.
 - *Processes the record immediately:* This minimizes delay, allowing for quick data handling.
 - *Forwards the output to the next node:* After processing, data is sent to the subsequent node, maintaining the flow.
- **Pros**
 - *Achieves extremely low latency:* The system is capable of responding in sub-millisecond times, making it highly efficient for real-time applications.

- **Cons**
 - *Poor fault tolerance*: The system’s reliability can be compromised unless additional nodes or redundant paths are implemented for failover.
 - *Sensitive to stragglers*: If a node processes data slower than others, it can bottleneck the entire system, delaying the overall processing pipeline.

16 / 21: Micro-Batch Stream Processing

Micro-Batch Stream Processing

- Break continuous stream into **small batches** (e.g., 1-second windows)
 - Implemented in Spark Streaming (aka “DStreams”)
- **Pros**
 - Recover from failures and stragglers with task scheduling
 - Schedule the same task multiple times
 - Deterministic tasks
 - Exactly-once processing
 - Consistent API: same semantics as RDDs
 - Fault-tolerance
- **Cons**
 - Higher latency
 - E.g., seconds



16 / 21

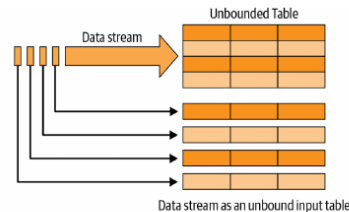
- **Micro-Batch Stream Processing**
 - This approach involves dividing a continuous data stream into *small batches*, such as 1-second windows. This method is implemented in systems like Spark Streaming, also known as “DStreams.”
- **Pros**
 - **Recover from failures and stragglers with task scheduling**
 - The system can handle failures by scheduling the same task multiple times, ensuring reliability.
 - **Deterministic tasks**
 - Provides *exactly-once processing*, meaning each piece of data is processed only once, avoiding duplicates.
 - Offers a consistent API with the same semantics as RDDs (Resilient Distributed Datasets), which simplifies development.
 - Built-in *fault-tolerance* ensures that the system can recover from errors without data loss.
- **Cons**
 - **Higher latency**
 - Processing in micro-batches can introduce latency, often in the range of seconds,

which might not be suitable for applications requiring real-time processing.

17 / 21: Spark Structured Streaming

Spark Structured Streaming

- Unified DataFrame/SQL-based model for both batch and streaming
- System manages state, fault tolerance, incremental computation, and late data
- Streaming table abstraction
 - Conceptually an unbounded table continuously appended with new rows
 - At time T , equivalent to a static DataFrame of all rows up to T



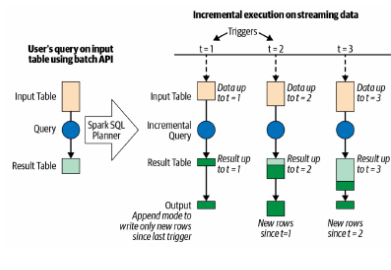
17 / 21

- **Unified DataFrame/SQL-based model for both batch and streaming**
 - Spark Structured Streaming provides a consistent way to handle both batch and streaming data using the same DataFrame and SQL APIs. This means you can apply the same operations and queries to both types of data, simplifying the development process.
- **System manages state, fault tolerance, incremental computation, and late data**
 - The system automatically handles complex tasks such as maintaining state across streaming operations, ensuring fault tolerance, and performing computations incrementally. It also deals with late-arriving data, ensuring that your results are accurate and up-to-date.
- **Streaming table abstraction**
 - *Conceptually an unbounded table continuously appended with new rows:* This abstraction allows you to think of streaming data as a table that is constantly growing as new data arrives.
 - *At time T , equivalent to a static DataFrame of all rows up to T :* At any given moment, the streaming table can be viewed as a snapshot of all the data received up to that point, similar to a static DataFrame. This makes it easier to reason about the data and apply transformations or queries.

18 / 21: Incrementalization

Incrementalization

- Framework identifies necessary state across micro-batches
- Uses DAG analysis to compute updated results from prior state
- Developers specify trigger conditions for updates
- Results updated incrementally as events arrive



18 / 21

- **Framework identifies necessary state across micro-batches**
 - The framework is designed to handle data in small, manageable chunks called micro-batches.
 - It keeps track of the necessary state, which means it remembers what data has been processed so far.
 - This helps in efficiently updating results without reprocessing all data.
- **Uses DAG analysis to compute updated results from prior state**
 - DAG stands for Directed Acyclic Graph, a structure used to model the flow of data.
 - By analyzing the DAG, the system can determine how to update results based on previously processed data.
 - This method ensures that only the necessary computations are performed, saving time and resources.
- **Developers specify trigger conditions for updates**
 - Developers can set specific conditions that determine when the system should update the results.
 - These triggers can be based on time intervals or specific events.
 - This flexibility allows for tailored data processing that meets specific needs.
- **Results updated incrementally as events arrive**
 - As new data comes in, the system updates the results incrementally.
 - This means only the new data is processed, and the existing results are adjusted accordingly.
 - Incremental updates are efficient and allow for real-time data processing, which is crucial for streaming applications.

19 / 21: Triggering Modes

Triggering Modes

- Indicate when to process newly available streaming data
- **Default**
 - Process micro-batch after previous completes
- **Trigger interval**
 - Specify fixed interval for each micro-batch
 - E.g., “every 10 minutes”
- **Once**
 - Wait for external trigger
 - E.g., “at end of day”
- **Continuous (experimental)**
 - Process data continuously
 - Not all operations available
 - Lower latency

19 / 21

- **Triggering Modes:** These are methods to determine when a streaming data processing system should handle new data. They help manage how and when data is processed in a streaming environment.
- ****@Default@**:**
 - This mode processes a new batch of data as soon as the previous batch is completed. It’s a straightforward approach that ensures data is processed as quickly as possible without any delay between batches.
- **@Trigger interval@:**
 - Here, you set a specific time interval for processing each batch of data. For example, you might choose to process data every 10 minutes. This allows for predictable processing times and can help manage system resources effectively.
- ****@Once@**:**
 - This mode waits for an external signal to start processing. For instance, you might choose to process data only at the end of the day. This can be useful for scenarios where data needs to be processed at specific times or under certain conditions.
- ****@Continuous (experimental)@**:**
 - In this mode, data is processed continuously, which can lead to lower latency, meaning data is processed almost in real-time. However, not all operations are supported in this mode, as it is still experimental. This approach is beneficial when immediate data processing is crucial, but it may require more advanced handling and resources.

20 / 21: Saving Modes

Saving Modes

- Indicate when to save results and where
 - Each time the result table updates, write to an external file system
 - E.g., HDFS, AWS S3, or a database (e.g., MySQL, Cassandra)
- **Append mode**
 - Append new rows since the last trigger
 - Use when existing rows don't change
- **Update mode**
 - Write updated rows since the last trigger
 - Update in place
- **Complete mode**
 - Write the entire updated result table
 - General but expensive

20 / 21

- **Saving Modes**
 - *Indicate when to save results and where:* This point is about deciding the timing and location for saving your data. It's crucial to have a strategy for saving data to ensure that you don't lose important information and that you can access it later for analysis or reporting.
 - *Each time the result table updates, write to an external file system:* This means that every time your data table gets new information or changes, you should save it to an external storage system. Examples include **HDFS** (Hadoop Distributed File System), **AWS S3** (Amazon Web Services Simple Storage Service), or a database like **MySQL** or **Cassandra**. These systems are designed to handle large amounts of data efficiently.
- **Append mode**
 - *Append new rows since the last trigger:* In this mode, you only add new data that has come in since the last time you saved. This is useful when you are continuously collecting new data and don't need to change the existing data.
 - *Use when existing rows don't change:* This mode is ideal when your data is mostly static, and you are only interested in adding new entries without modifying the old ones.
- **Update mode**
 - *Write updated rows since the last trigger:* Here, you save only the rows that have changed since the last time you saved. This is efficient when you need to keep your data up-to-date without rewriting everything.
 - *Update in place:* This means you directly modify the existing data with the new changes, which can be more efficient than rewriting the entire dataset.
- **Complete mode**

- *Write the entire updated result table:* This mode involves saving the whole data table every time there is an update. It's a comprehensive approach but can be resource-intensive.
- *General but expensive:* While this method ensures that you have the most complete and up-to-date data, it can be costly in terms of storage and processing power, especially with large datasets.

21 / 21: Spark Streaming “Hello world”

Spark Streaming “Hello world”

- `lines` is a `DataStreamReader`
 - Unbounded `DataFrame`
 - Set up reading but doesn't start reading
- `words` splits data into words
- `counts` is a streaming `DataFrame`
 - Running word count
- `select()`, `filter()` are stateless transformations
- `count()` is a stateful transformation
- Configuration
 - How to write processed output
 - Where to write (e.g., `console`)
 - How to write (e.g., `complete` for updated word counts)
 - When to trigger computation (e.g., every 1 second)
 - Where to save metadata for exactly-once guarantees and failure recovery
- `start()` processing (non-blocking)
 - `awaitTermination()` blocks until data is available

```
from pyspark.sql.functions import *
spark = SparkSession...
lines = (spark
    .readStream.format("socket")
    .option("host", "localhost")
    .option("port", 9999)
    .load())

words = lines.select(split(col("value"), "\\s").alias("word"))
counts = words.groupBy("word").count()
checkpointDir = "...
streamingQuery = (counts
    .writeStream
    .format("console")
    .outputMode("complete")
    .trigger(processingTime="1 second")
    .option("checkpointLocation", checkpointDir)
    .start())
streamingQuery.awaitTermination()
```

21 / 21

- **`lines` is a `DataStreamReader`**
 - Represents an *unbounded `DataFrame`*, meaning it can handle continuous data streams.
 - It sets up the reading process but doesn't start reading data immediately.
- **`words` splits data into words**
 - This step processes the incoming data by splitting each line into individual words.
- **`counts` is a streaming `DataFrame`**
 - Performs a running word count, continuously updating as new data arrives.
- **`select()`, `filter()` are stateless transformations**
 - These operations do not depend on previous data; they process each piece of data independently.
- **`count()` is a stateful transformation**
 - This operation maintains state over time, keeping track of word counts as data streams in.
- **Configuration**
 - Specifies how to write the processed output, such as writing to the `console`.
 - Defines the output mode, like `complete`, which updates word counts.
 - Sets the trigger for computation, e.g., every 1 second.
 - Determines where to save metadata to ensure exactly-once processing and recovery from

failures.

- **start() processing (non-blocking)**
 - Begins the data processing stream without blocking other operations.
 - **awaitTermination()** blocks the process until data is available, ensuring the program waits for streaming data to process.